

Feature: Contact and enquiry

From build-plan: feature 9 Status: verified Branch: `feature/contact-and-enquiry`

Goal

A `/contact` page that turns "I have a question" into a text message, with a short Resend-backed form as the fallback and a clear path for the three treatments that have no published price. Every Book now button on the site already points at `/contact`, so today they all land on a 404; after this feature they land on a page that answers the question and gets the visitor to her phone.

Text is the primary channel because she is usually mid-treatment: a text is answered between clients, a call goes to voicemail. The page says so.

Design reference

`prototypes/contact.html`, sections in order: hero with crumbs, the three ways (Text largest and tinted, Call, Email), hours beside the form, the priced-at-consultation list with per-treatment "Ask about pricing" text links, and location. Tokens are already in `globals.css`. The open-now pill, map, directions and parking in that prototype belong to feature 8 and stay out.

In scope

- **Route** `app/contact/page.tsx`, static, with metadata title "Contact" and a description drawn from the page lede.
- **Hero**: breadcrumb (Home / Contact), eyebrow "Get in touch", h1 "Text us. It's the fastest way.", the prototype lede, and a photo from an existing `IMAGES` slot. Watermark orchid as on the home hero.
- **Three ways**, all reading `siteConfig`:
 - Text, primary, tinted card, `sms`: link with a prefilled body "Hi! I'd like to ask about ", label Recommended.
 - Call, `tel`: link, with a line pointing at the hours.
 - Email, `mailto`: link to `siteConfig.email`.
- **Hours column**: the seven-row table already on the home page, extracted into a shared component so both pages read one markup, plus the address lines. No open-now state here.
- **Form**, the fallback: Name; Phone or email; What's it about, a native select; Message; a hidden honeypot. Submits to a Server Action that sends one plain-text email through Resend to `siteConfig.email`. Success replaces the form with a confirmation. Failure shows an inline message

with the text and call links, because a visitor who cannot reach the form must still be able to reach the business.

- **Topic options**, generated, not typed: "A question about a treatment", then "Pricing for <name>" for every service in `SERVICES` with `price: null` (today Fractional Microneedling, IPL Treatment, Laser Hair Removal), then "Am I a candidate for this", "Gift cards", "Something else".
- **Priced at consultation** section: the same unpriced services from the data, each with a name, its category label, and an "Ask about pricing" `sms:` link whose body is "Hi! Can I get pricing for <name>?". A closing line pointing at the free consultation with a link to the form.
- **Ask-about-pricing entry from elsewhere:** `/contact?treatment=<slug>`. When the slug matches a service, the form's topic preselects "Pricing for <name>" for an unpriced service or "A question about a treatment" otherwise, and the message starts "Hi, I have a question about <name>.". An unknown slug is ignored. Feature 5's treatment pages will link here; the contract is defined now so they do not have to invent it.
- **Text link helper** so no component hand-builds `sms:` URLs.
- **Header and footer:** unchanged. The header CTA already points here.

Out of scope

- Open-now state, map embed, directions, parking (feature 8, which may also add a `/visit` route or fold into this page).
- JSON-LD `ContactPage` schema, Open Graph image (feature 10).
- Booking (feature 11). "Book a consultation" links on this page point at the form section until then.
- An HTML email template. Plain text carries the four fields and needs no escaping rules.
- Storing enquiries anywhere. The overview says nothing about a visitor is persisted; the email is the record.
- Rate limiting beyond the honeypot. Nothing in the packet asks for it and a Server Action has no public URL.
- Toast notifications. No toast component is installed; status is inline.

Build loop

`workflow.stepReview` is `feature`: build all steps, then present one review packet.

`workflow.checkpointCommits` is `disabled`: no commits from `/implement`; `/complete` makes the single feature commit. No Verify command is declared, so the gate is `npm run lint` and `npm run build`. UI evidence is `when-available`: the dev server on port 3001 is running, so each step's done-when includes what to observe there.

Build steps

- **1. Helpers and the shared hours table.** Add `lib/contact.ts` with `smsLink(message)`, which returns the `sms:` number from `siteConfig` followed by `?&body=` and the URL-encoded message (that form opens correctly on both iOS and Android), `pricingMessage(service)`, `getUnpricedServices()` (services with `price === null`, in data order) and `CONTACT_TOPICS` built from them. Extract the hours table from `components/home/Location.tsx` into `components/business/HoursTable.tsx` and use it there. **Done when:** `npx tsc --noEmit` passes, `npm run build` passes, and the home page hours table renders exactly as before at 375px.
- **2. The page.** Add `app/contact/page.tsx` with metadata, the hero, the three ways, the hours column with address, the priced-at-consultation list, and a placeholder heading where the form goes. Sections use the existing `Section`, `Container`, `Eyebrow`, `Lede`, `Photo`, `Watermark` and button variants; the sand tone for the hours-and-form band as in the prototype. **Done when:** `/contact` renders at 375px and 1280px in both themes; the Text card's href starts with the `sms:` number and `?&body=Hi!`, Call is the `tel:` number, Email is `mailto:` plus `siteConfig.email`; the pricing list shows the three unpriced services with distinct sms bodies; the header Book now and every home page CTA now land on this page.
- **3. Form and Server Action.** Add the shadcn `input`, `label` and `textarea` files (`npx shadcn add input label textarea`; `@base-ui/react` is present so no package should install; stop if one does). Add `lib/contactSchema.ts` (zod), `actions/contact.ts` ("use server"), and `components/contact/ContactForm.tsx` ("use client", react-hook-form with the zod resolver, no shadcn Form wrapper per the coding standards). The form reads the `treatment` prefill from `window.location` after mount, so the page stays static with no Suspense boundary. **Done when:** submitting empty shows a message under each invalid field, each input is linked to its message by `aria-describedby` and marked `aria-invalid`, and focus moves to the first invalid field; a valid submit with `RESEND_API_KEY` unset shows the inline failure with working text and call links; with the key set the confirmation replaces the form and focus moves to its heading; `/contact?treatment=laser-hair-removal` preselects "Pricing for Laser Hair Removal"; `npm run lint` and `npm run build` pass.

Files / areas

Path	Change	Notes
<code>app/contact/page.tsx</code>	new	static route, metadata, section composition
<code>components/contact/ContactForm.tsx</code>	new	client form, the only client component on the page
<code>components/contact/Ways.tsx</code> , <code>PricingEnquiry.tsx</code>	new	server sections
<code>components/business/HoursTable.tsx</code>	new	extracted from <code>Location.tsx</code> , used by both
<code>components/home/Location.tsx</code>	edit	renders <code>HoursTable</code>
<code>components/ui/input.tsx</code> , <code>label.tsx</code> , <code>textarea.tsx</code>	new	shadcn files
<code>lib/contact.ts</code>	new	<code>smsLink</code> , <code>pricingMessage</code> , <code>getUnpricedServices</code> , <code>CONTACT_TOPICS</code>
<code>lib/contactSchema.ts</code>	new	zod schema shared by form and action
<code>actions/contact.ts</code>	new	Server Action, Resend send
<code>.env.example</code>	edit	<code>RESEND_FROM</code> line; the file is git-ignored, so this lands only locally

Data / contracts

Form values, validated by one zod schema on both sides:

| Field | Rule | |---|---| | `name` | string, trimmed, 2 to 80 characters | | `contact` | string, trimmed; valid when it parses as an email or contains at least 10 digits | | `topic` | one of `CONTACT_TOPICS`, default the first | | `message` | string, trimmed, 10 to 2000 characters | | `company` | honeypot, must be empty; a filled honeypot returns success without sending |

Action result: `{ success: true } | { success: false; error: string }`. The error string is the same for every server-side failure: missing key, Resend error, thrown exception. It names the phone number so the visitor has somewhere to go. Validation failures on the server return `{ success: false, error: "Invalid submission." }`; the client never shows that path because it validates first.

Email: plain text. `to` is `siteConfig.email`. `from` is `process.env.RESEND_FROM`, falling back to `onboarding@resend.dev`, which Resend only delivers to the account owner's address and is therefore a local testing sender, not a production one. `replyTo` is the contact value when it parsed as an email. Subject `New enquiry from <name>`. Body lines: Name, Contact, Topic, blank, Message. No HTML, so user text is never rendered.

Query contract: `?treatment=<service slug>`. Read on the client only, after mount.

smsLink output: the `sms:` number, then `?&body=`, then the URL-encoded message.

Testing

No test runner is configured, so the step done-whens rely on the build and the running dev server. If `/tests` is run later, the first candidates are `smsLink`, `getUnpricedServices`, `CONTACT_TOPICS` order, and the `contactSchema` accept and reject cases (email vs phone in `contact`, the honeypot).

Notes for the AI

- `siteConfig` already holds phone display, `tel:`, `sms:`, email, address and hours. Read them; never type a number.
- Base UI wraps things in ways that fight the global heading styles; the header fix hit this twice. Prefer the shadcn `input`, `label`, `textarea` files and a native `<select>` styled with the same classes, which also gives phones their native picker.
- Keep the page a server component. `ContactForm` is the one client island. It reads the query from `window.location` in an effect rather than `useSearchParams`, because the latter needs a Suspense boundary that the Next 16 dev server streams unreliably, which hid the form during review.
- The address rows are the same three lines as `Location.tsx`; the postal code question is recorded there and is not this feature's to resolve.

- `.env.example` is git-ignored in this repo (`.env*`), so document `RESEND_FROM` in the archive walkthrough too.

Open questions

- **Sender domain.** Resend only sends from a verified domain, and she owns none yet (overview, Open questions). Until a domain exists the form sends through the Resend onboarding sender, which delivers only to the Resend account owner. This does not block the build: the action is written to the final shape and only the `RESEND_FROM` value changes when the domain lands. It does block calling the form production-ready, and `/release` should list it.

Implementation walkthrough

Helpers (`lib/contact.ts` , `lib/contactSchema.ts`) - `smsLink` builds the `sms:` URL with the `?&body=` form, the one spelling that opens a prefilled text on both iOS and Android.

`getUnpricedServices` filters `SERVICES` on `price === null` and drives both the topic list and the pricing section, so adding a price to a service removes it from both without touching the page. The zod schema is shared by the form and the action; the honeypot field is `optional()` rather than `rejected` so a filled one reaches the action and is answered with success, never with a validation error a bot could learn from.

Shared hours table (`components/business/HoursTable.tsx`) - the seven-row table lifted out of the home page `Location` section unchanged, now rendered by both pages. Feature 8 adds the open-now state here.

Page (`app/contact/page.tsx` , `components/contact/Ways.tsx` , `PricingEnquiry.tsx`) - a static route composed from the existing section primitives. The three ways are one `<a>` per card so the whole card is the tap target; Text is the tinted copper card with the Recommended tag. The photo slots reuse `consultation` and `catSkin` rather than adding images. The pricing section links "Send a message" to `#form` because booking is feature 11.

Form (`components/contact/ContactForm.tsx`) - react-hook-form with the zod resolver and no shadcn Form wrapper, per the coding standards. Field errors are wired with `aria-invalid` and `aria-describedby` ; RHF focuses the first invalid field on submit, and a `useEffect` moves focus to the confirmation heading when the result flips to success (an effect rather than a ref read inside the handler, which the project's lint rejects). The topic select is a native `<select>` styled with the input classes, so phones get their own picker. The failure message is short and the form appends the text and call links, since the visitor the form let down still needs a way in.

The prefill and the dev server - the first cut read `?treatment=` with `useSearchParams` inside a `Suspense` boundary. That is correct for a static route and worked on `next start` , but the Next 16 dev server never streamed the boundary in on any query-string load and often stalled on plain loads, so the form box showed only its heading while Frank reviewed. The final cut reads `window.location.search` in

an effect after mount: no hook, no boundary, the form is in the server HTML on every load, and the prefill still lands because the effect runs on the client. Recorded so the next page that reads a query does the same.

Action (`actions/contact.ts`) - a Server Action, so there is no public URL to post to. Every failure past validation returns the same short message. `from` is `RESEND_FROM` or Resend's onboarding sender, `to` is `siteConfig.email`, `replyTo` is the visitor's contact when it parsed as an email. Plain text only, so nothing the visitor typed is ever rendered as markup. `.env.example` gained a `RESEND_FROM` line but the file is git-ignored, so set it by hand: the sender must be at a domain verified in Resend, and until she owns one the form only delivers to the Resend account owner.

Verified - `npx tsc --noEmit`, `npm run lint`, `npm run build` clean, three static routes. On the dev server: title "Contact | Face and Body Wellness Centre", seven hours rows, three pricing rows, seven topics, and the sms, tel and mailto hrefs read from the config. Empty submit marks three fields invalid with linked messages and focuses Name. A honeypot submit replaced the form with "Sent. We reply within a day." and focused that heading. Two valid submits with the local key reached Resend and were rejected because the onboarding sender cannot deliver to her address; that is the failure path working and no email was sent. `/contact?treatment=laser-hair-removal` preselects "Pricing for Laser Hair Removal" and starts the message; an unknown slug is ignored. Mobile at 375px has no horizontal overflow. A real successful send remains unverified until a domain is verified in Resend.

Findings

09/F-01 [P1] accepted - The contact Server Action has no rate limit or abuse guard

File: `actions/contact.ts:18` **Found:** 2026-09-18 by /audit (scope: changed; lens: security) **Why it matters:** `submitContact` is publicly invocable with no authentication and no throttle. The only guard is the `company` honeypot, which a scripted call bypasses by not sending the field. Every accepted call sends mail through Resend to her Gmail, so an unbounded flood both fills the inbox the site funnels into and burns the Resend quota, after which genuine enquiries fail too. **Resolution: Accepted 2026-09-18 by Frank.** Reason: no traffic yet and the form cannot deliver until she has a verified domain, so Vercel Firewall at launch rather than a hand-rolled limiter in process.

An in-process limiter was built and then reverted. It worked, holding the email budget across 20,000 rotated IPs, but it introduced 09/F-07 and that is the durable lesson: per-instance state is the wrong layer for this on serverless.

09/F-07 [P1] closed - The rate limiter rescans an unbounded map on every request

File: `lib/rateLimit.ts:30` (file no longer exists) **Found:** 2026-09-18 by /audit (scope: changed; lens: performance) **Why it matters:** `prune` walked the whole map on every call and `hit` inserted an entry per new IP before the overall ceiling was consulted, so a caller rotating IPs grew the map without limit and made each later request dearer. Measured: 2,000 IPs 21 ms, 4,000 65 ms, 8,000 194 ms, 16,000 745

ms, 200,000 over two minutes. Doubling the input quadrupled the work. 119 bytes retained per blocked request, about 113 MB at a million IPs. The guard written to stop a denial of service was itself the amplifier. **Resolution:** 2026-09-18 - resolved by reversal. `lib/rateLimit.ts` deleted and `actions/contact.ts` restored, so the unbounded map exists nowhere in the project. Typecheck, lint and build clean afterwards.

09/F-08 [P2] closed - The per-IP limit rested on an unchecked header assumption

File: actions/contact.ts:23 (code no longer exists) **Found:** 2026-09-18 by /audit (scope: changed; lens: security) **Why it matters:** `clientIp` trusted `x-vercel-forwarded-for` on the grounds that the edge sets it and a client cannot forge it. That was asserted, never verified. If a client could set it, the per-IP limit fell to one header value. **Resolution:** 2026-09-18 - moot. `clientIp` was removed with the limiter. If rate limiting returns through Vercel Firewall, identifying the caller is Vercel's job and the question does not come back.